

Mibo Smart — Documento de Produto

Case Mibo Smart — Documento de produto e arquitetura

Rascunho para o PDF de entrega (quarta 23/09). Itens marcados `TODO` dependem da implementação. Referências: `docs/adr/` (decisões), `docs/specs/SPEC.md` (critérios de aceite), `docs/api-contract.md` (contrato observado), `AI-LOG.md` (uso de IA), guias práticos em `docs/guides/` ([token](#), [leitura do Swagger](#), [como rodar](#)).

1. Contexto e objetivo

- Case técnico da plataforma Open Casa Inteligente para a vaga de Analista de Desenvolvimento de Produto II — Android.
- Entregar um app Kotlin Multiplatform (Android principal, iOS como prova do compartilhamento) que consome a Open Casa Inteligente: token de acesso, lista de dispositivos, vídeo ao vivo de câmeras, controle de fechadura.
- O enunciado pede “como você pensa, prioriza, comunica e codifica” — este documento é a parte “pensa e comunica”; o repositório (Issues → PRs → ADRs) é a trilha do “prioriza e codifica”.

2. Leitura do problema

- App conectado a hardware real: os estados que importam não são só loading/erro, mas “comando enviado e não confirmado”, “câmera offline”, “abertura remota desabilitada”, “token expirou no meio da sessão”, “cota de streaming esgotada”.
- O contrato da API tem particularidades que definem a arquitetura (ver §6): HTTP sempre 200, dois formatos de envelope, token rejeitado indistinguível de erro genérico, orçamento finito de requisições.
- Três entregáveis: repositório versionado, este documento, log do uso de IA.

3. Escopo e priorização

Onda	Escopo	Requisitos	Racional
0	<i>Walking skeleton</i> : repositório, CI, esqueleto de módulos, specs, ADRs, testes de arquitetura	—	a trilha começa antes do código
1	Sessão/token (tela, validação, expiração) — constrói client HTTP, envelope, erros tipados e cofre por necessidade própria	RF01, RF04	primeira fatia vertical; não existe “fatia de fundação”
2	Em paralelo: lista + filtro + paginação + estados · vídeo ao vivo · fechadura (status, abrir/fechar, pré-condição remota, volume)	RF02, RF03, RF04, RF05, RF06, RF07, RF08	as três não dependem entre si; vídeo e fechadura são o “hardware real”
3	Histórico da fechadura, renovação de token, iOS com a lista, módulo Java, R8 no release, este PDF	RF09, ★ Java	só após o núcleo funcionar no Android

- Circuit breaker: must-have para a apresentação são as ondas 0–2. Se o prazo apertar, a onda 3 encolhe nesta ordem: módulo Java (ADR-007) → histórico (RF09) → fallback WebView do vídeo → R8. Vídeo e fechadura não são cortáveis.
- Fora de escopo por decisão: lâmpadas, sensores, gravações, senhas de fechadura, criação de conta.

3.1 Visão de produto: o que os usuários reais pedem

Antes de escrever as issues, li o que quem usa o app oficial da plataforma diz nas lojas (docs/research/user-feedback.md : 250 avaliações recentes da App Store, 20 da Play, Reclame Aqui e o fórum oficial; leitura em 20/09/2026). Nota agregada 4,8★ nas duas lojas, mas as avaliações recentes *com texto* são majoritariamente 1★ — a leitura é sobre o texto.

O que mais dói (≈ menções em 270)	O que o case faz a respeito
Vídeo ao vivo trava em “9x %” sem erro nomeado (≈50)	Etapas visíveis, timeout de 20 s com mensagem , retry e fallback web — nunca um spinner infinito (SPEC U1, V3–V5)
Propaganda e push de marketing num app de segurança (≈45)	Zero banner, pesquisa, interstitial ou notificação; abre direto na lista, câmera em dois toques (U2, U7)
Câmera “offline do nada”, sem explicação (≈20)	Indicador + “visto pela última vez há X” na lista e na fechadura (U3)
Fechadura: histórico chega atrasado e sem nome; volume que “sumiu” (≈10)	Comando → confirmação explícita; histórico com quem e quando; volume com rótulos (L3–L7, U4)
Sessão que “perde a senha” e obriga a reinstalar (≈6)	Expiração explicada e retorno à tela anterior após o novo token (U5)
Erros genéricos (“Erro desconhecido”)	Toda mensagem diz a causa e oferece uma ação; nada de código HTTP (U6)

Fora do escopo do case, mas registrado como próximos passos de produto: gravações/timeline/SD (segundo maior cluster), notificações de dispositivo com granularidade, um app e uma conta para todas as linhas, e um relato de segurança da fechadura (senha excluída continuando ativa) que merece tratamento de incidente. O detalhe está no §5–6 do documento de pesquisa.

4. Arquitetura

```

:shared:domain  modelos, contratos (DeviceRepository, LockRepository, StreamingRepository,
                  SecureTokenStore), erros e resultados por caso de uso – sem dependências
:shared:data    implementação do parceiro: Ktor, DTOs, EnvelopeReader, mapeadores, SQLDelight;
                  cofre do token em `platform.vault` (Keystore no androidMain, Keychain no iosMain)
:shared:app     casos de uso, ViewModels, UI Compose Multiplatform por feature, módulos Koin;
                  superfície de vídeo `camera.platform.LiveVideoPlayer` (Media3 no androidMain,
                  WKWebView no iosMain)
:androidApp    Activity + Application (só plumbing, nenhum `actual`)
iosApp         host SwiftUI (Swift não hospeda `actual` Kotlin)
:legacy-catalog módulo Java pequeno (SDK legado simulado) – onda 2, cortável

```

- `expect/actual` só existe em pacotes chamados `platform` dentro do módulo dono da abstração (regra 8 dos testes de arquitetura). O player é superfície de UI, não contrato de domínio (ADR-005).
- Dependências apontam para dentro; `:shared:domain` não conhece Ktor, Koin, Compose nem SQLDelight (ADR-001). “Suporte a múltiplos parceiros” = um segundo módulo `data` implementando os mesmos contratos (ADR-004).
- Apresentação: MVVM com um estado imutável por tela (`StateFlow`), ações como funções suspensas, estados de hardware modelados como tipos selados (ADR-003).
- Erro é valor: a camada de dados lança exceções tipadas; cada caso de uso devolve o resultado da sua intenção (`Loaded` | `Empty` | `TokenRejected` | `Offline` | `Failure`) (ADR-002).

- `TODO` diagrama de sequência do vídeo (criar sessão → player em < 15 s → encerrar sessão) e da fechadura (comando → confirmação → estado).

5. Stack e justificativas

Camada	Escolha	Por quê (resumo)
Linguagem/UI	Kotlin 2.4 · Compose Multiplatform 1.11	estado da arte JetBrains; Compose iOS estável
Rede	Ktor 3.5	cliente multiplataforma oficial
Persistência	SQLDelight 2.3	maduro em KMP; Room 2.8 seria equivalente; Room 3.0 ainda alpha
DI	Koin 4.2	modularização por camada; Metro 1.0 (compile-time) citado como fronteira
Concorrência	Coroutines + Flow/StateFlow	structured concurrency; cancelamento propagado
Vídeo	Media3 (Android) · WKWebView em <code>monitor_url</code> (iOS) · WebView fallback no Android	stream é fMP4 sobre HTTP; AVPlayer não reproduz sem HLS; VLCKit descartado (tamanho + interop fora do prazo)
Segurança	Keystore / Keychain via interop direto	controle fino de acessibilidade do segredo
Testes	Mokkery · kotlin-coroutines-test · Turbine	mocks em Kotlin/Native; <code>state.value + advanceUntilIdle()</code> para o estado da tela, Turbine só para eventos one-shot

6. O contrato da API e como ele moldou o código

- HTTP sempre 200 → o “status” real é lido do corpo; `EnvelopeReader` aceita os dois formatos.
- Token inválido devolve “Erro desconhecido...” → regra de negócio no caso de uso, não no client.
- Orçamento de ~300 requisições (valor lido em 20/09/2026) → cache local da lista, classificação por `modelo` antes de `funcoes`, zero polling, retry só para falha de rede (ADR-006).
- Fechadura é sub-dispositivo: `ns` composto `lock_hub_idProdutoHub`; abertura remota precisa estar habilitada; `volume/v1` exige `productId` mas o campo é `idProduto` (enviado em dobro).
- Stream expira em 15 s sem player; sessão encerrada ao sair, inclusive sob cancelamento.
- `TODO` tabela final “contradição do contrato → onde está tratada no código (arquivo/teste)”.

7. Estados de hardware tratados

- Fechadura: trancada · destrancada · comando em andamento · abertura remota desabilitada · offline · estado não confirmado (com ação “Verificar”, sem polling).

- Câmera: criando sessão · ao vivo · reconectando (n/3) · expirada · cota esgotada · offline · falha com fallback web.
- Sessão: sem token · válida (expira em ...) · expirada (rota para a tela de token com mensagem específica).
- `TODO` capturas de tela de cada estado.

8. Segurança

- Token só em Keystore/Keychain (`ThisDeviceOnly`), nunca em preferências simples, logs ou UI (só o sufixo).
- `Authorization` sanitizado no logger HTTP; CI falha se o padrão de token aparecer no repositório.
- Nenhuma credencial versionada (regra do enunciado). Detalhes: ADR-008.

9. Qualidade, testes e método

- Metodologia: SDD leve — spec com critérios EARS/Gherkin (`docs/specs/SPEC.md`) → ADRs → Issues no GitHub (uma por fatia vertical) → um PR por Issue com o “porquê” na descrição.
- TDD cirúrgico nas regras críticas: paginação/filtro, tradução de erro/token, máquina de estado da fechadura, política de retry do vídeo. UI e player fora do TDD.
- Uso de IA documentado em `AI-LOG.md` — inclusive o que a IA errou e como foi corrigido.
- `TODO` números finais: testes por módulo, cobertura das regras críticas, resultado do CI.

10. Como rodar

- Android: `./gradlew :androidApp:assembleDebug` e instalar; colar o token temporário na tela inicial.
- iOS: abrir `iosApp` no Xcode (macOS) — `TODO` confirmar passos após onda 3.
- Testes: `TODO` comandos finais por módulo (ver `docs/guides/running.md`).
- Token: gerado na plataforma Open Casa Inteligente → Contas → Token Temporário (validade 2 h) — ver `docs/guides/token.md` .

11. O que ficou de fora e por quê

- `TODO` preencher na quarta com o que realmente não entrou (e o motivo: prazo, cota, contrato).
- Renovação de token (S10): planejada para a onda 3 e primeira na lista de corte; o endpoint `renovarToken` só é chamado de verdade no início da onda 3, porque rotaciona o token em uso.

12. Roteiro da apresentação (20 min)

- 3 min contexto e leitura do problema · 5 min demo (token → lista → câmera → fechadura → erro de token) · 7 min arquitetura e decisões (ADRs 1–5) · 3 min contrato da API e o que ele forçou · 2 min método e uso de IA (um erro corrigido, de verdade).